

SIKSHA 'O' ANUSANDHAN (DEEMED TO BE UNIVERSITY)

Admission Batch: 2023-2027

Session: 2026-2027

Laboratory Record Machine Learning Projects with Python (CSE 4192)

Lab Assignment 01

Predicting Diabetes with Multilayer Perceptrons

Submitted by

1. Name: Subhashree Mohanty (Regd. No.: 2341013309)
2. Name: Manish Kumar Baitharu (Regd. No.: 2341002004)
3. Name: Omm Prakash Nayak (Regd. No.: 2341014065)
4. Name: Ayushman Sahoo (Regd. No.: 2341019113)



Centre for Artificial Intelligence & Machine Learning

Faculty of Engineering & Technology (ITER)

Jagamohan Nagar, Jagamara, Bhubaneswar, Odisha - 751030

1 Introduction

Diabetes is a common health condition where the body cannot control blood sugar levels properly. If it is not detected early, it can lead to serious health problems over time, such as damage to the eyes, kidneys, and heart. Doctors normally diagnose it using blood tests, but machine learning can help by looking at patterns in patient data and flagging people who might be at risk. In this project, we built a machine learning system that predicts whether a person is likely to have diabetes, using a few basic health measurements such as glucose level, BMI, and age.

We used the Pima Indians Diabetes dataset, a well-known dataset for this type of problem. We first trained a few standard machine learning models as a baseline, and then built and tuned a Multilayer Perceptron (MLP), which is a simple type of neural network. We compared all the models using proper evaluation metrics rather than accuracy alone, and picked the best one based on those results. Finally, we built a small web application using Streamlit so that a user can enter patient details and get a prediction along with the model's confidence.

This report walks through each step we followed- understanding the dataset, cleaning it, building the models, tuning the MLP, comparing the results, and deploying the final model.

This system is developed for academic and educational purposes and should not be used as a standalone medical diagnostic system. It is meant purely for academic learning and must not be used to actually diagnose diabetes in real patients.

2 Problem Statement, Aim and Objectives

2.1 Problem Statement

The goal is to build a system that can classify a patient as diabetic or non-diabetic based on their health data. Several standard machine learning models are compared with a Multilayer Perceptron, and the final model is chosen using proper evaluation results instead of accuracy alone.

2.2 Aim

To build, evaluate, tune, and deploy a Multilayer Perceptron model that predicts diabetes using the Pima Indians Diabetes dataset.

2.3 Objectives

- Understand and explore the dataset.
- Clean the data and handle missing or invalid values.
- Engineer a few useful features and scale the data.
- Train baseline models: Logistic Regression, KNN, SVM and Random Forest.
- Build and tune a Multilayer Perceptron.
- Compare all models using accuracy, precision, recall, F1-score and ROC-AUC.
- Select the best model and deploy it using Streamlit.
- Discuss the limitations of the project.

3 Dataset Description

3.1 Dataset Source

We used the Pima Indians Diabetes dataset, available on Kaggle at <https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database> [1]. It was originally collected by the National Institute of Diabetes and Digestive and Kidney Diseases, and the data-collection method is described by Smith et al. [2]. It contains health records of 768 female patients of Pima Indian heritage, all aged 21 or older. This is an important point to keep in mind — the dataset does not represent the general population, so our results should not be assumed to generalize to other groups such as men or younger patients.

3.2 Dataset Features

The dataset has **8 input features** and one target column called *Outcome*, which is 1 if the patient has diabetes and 0 if not. Out of 768 patients, 500 are non-diabetic and 268 are diabetic.

	Feature	Data Type	Description	Role
0	Pregnancies	Numerical	Number of pregnancies	Predictor
1	Glucose	Numerical	Plasma glucose level (mg/dL)	Predictor
2	BloodPressure	Numerical	Diastolic blood pressure (mm Hg)	Predictor
3	SkinThickness	Numerical	Triceps skin fold thickness (mm)	Predictor
4	Insulin	Numerical	2-hour serum insulin (μ U/mL)	Predictor
5	BMI	Numerical	Body mass index	Predictor
6	DiabetesPedigreeFunction	Numerical	Family history-based diabetes likelihood	Predictor
7	Age	Numerical	Patient age (years)	Predictor
8	Outcome	Binary (0/1)	Diabetes outcome: 1 = diabetic, 0 = non-diabetic	Target

Figure 1: Dataset Features

3.3 Target Variable

The target variable is *Outcome*, which represents whether or not a patient has diabetes. It is a binary classification variable with two possible classes: **0 (Non-Diabetic)** and **1 (Diabetic)**. A value of 0 indicates that the patient does not have diabetes, whereas a value of 1 indicates that the patient has diabetes. The dataset contains 500 non-diabetic patients and 268 diabetic patients.

3.4 Dataset Statistics

The dataset consists of **768 instances** and **9 attributes** in total. It contains 8 independent input features and 1 dependent target variable. All eight input features are numerical variables, while the target variable *Outcome* is binary. The dataset includes clinical and demographic information such as pregnancies, glucose level, blood pressure, skin thickness, insulin level, BMI, diabetes pedigree function, and age.

4 Exploratory Data Analysis

4.1 Descriptive Statistics

Descriptive statistics were performed to understand the distribution and variability of the dataset. The dataset contains 768 observations, with notable variation across features such as Glucose, BMI, Age, and Insulin. The summary statistics also helped identify unusual zero values and potential outliers for further analysis.

4.2 Missing Value Analysis

The dataset was examined to identify any missing values across the different features. No explicit missing values (NaN) were found in any of the columns. However, some features such as *Glucose*, *BloodPressure*, *SkinThickness*, *Insulin*, and *BMI* contain zero values, which are not practically meaningful for these medical measurements.

Therefore, these zero values were therefore considered as possible missing or unavailable measurements and considered them during the preprocessing stage. This helped us prepare the data properly before using it for model training.

4.3 Class Distribution

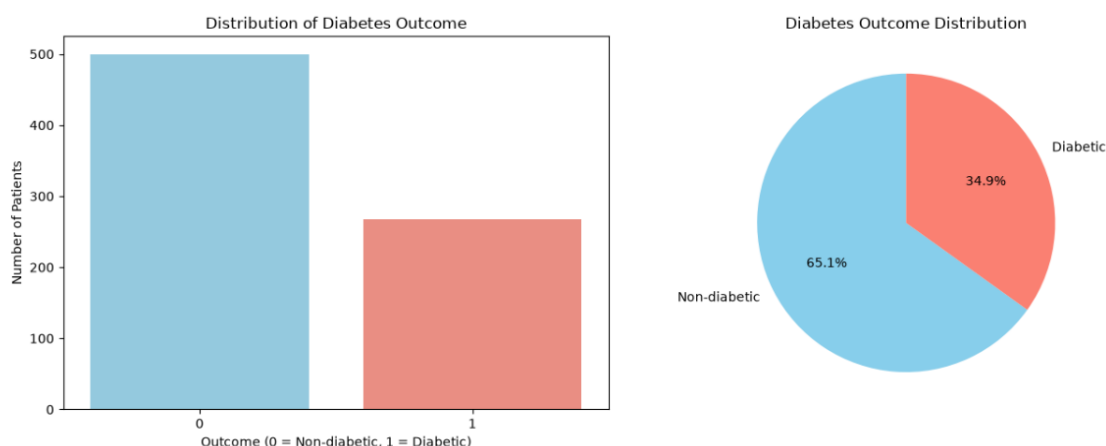


Figure 2: Number of diabetic and non-diabetic patients in the dataset.

Figure 2 shows that non-diabetic patients (about 65%) outnumber diabetic patients (about 35%). This class imbalance is one reason why we did not rely on accuracy alone when comparing models later.

4.4 Feature Distributions

Figure 3 shows how each feature is spread out. Most features look roughly bell-shaped, but Insulin and DiabetesPedigreeFunction are skewed — most values are small, but a few patients have unusually high values. This is expected for these kinds of medical measurements.

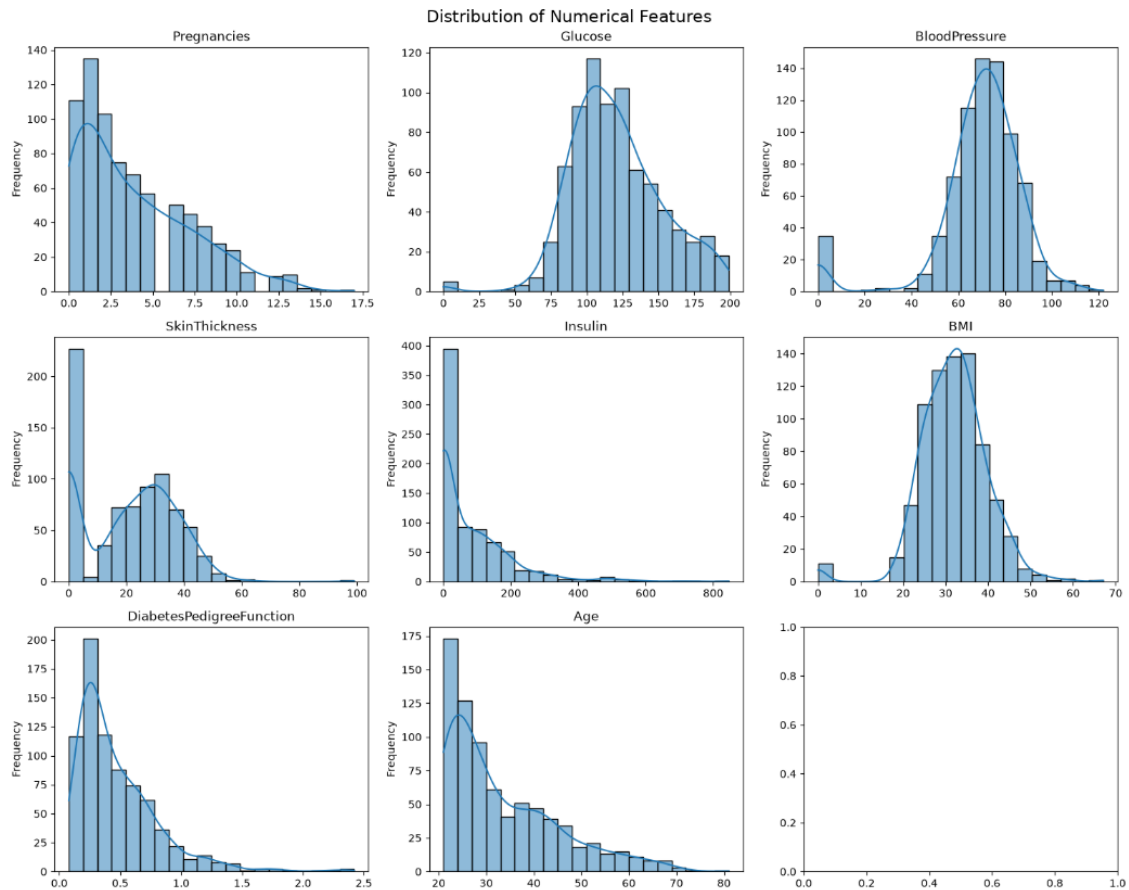


Figure 3: Distribution of each numerical feature.

4.5 Bivariate Analysis



Figure 4: Glucose, BMI, Age, and Blood Pressure compared between diabetic and non-diabetic patients.

Figure 4 compares four features across the two classes. Diabetic patients generally have higher Glucose and BMI, which matches what is already known about diabetes risk factors. Blood Pressure did not show as clear a difference between the two groups.

4.6 Correlation and Outliers

Figure 5 shows that Glucose has the strongest relationship with Outcome, followed by BMI and Age. No two features were too strongly related to each other, so we did not



Figure 5: Correlation between features and the target.

need to remove any feature for that reason.

We also checked for extreme values using the IQR method. Insulin and SkinThickness had the most extreme values, but we kept them because such values are realistic for patients with insulin resistance or obesity. Removing them could have thrown away useful information.

5 Data Preprocessing

5.1 Data Cleaning

The dataset was examined to identify duplicate records, missing values, and potentially invalid entries. No duplicate records were found. Zero values in Glucose, BloodPressure, SkinThickness, Insulin, and BMI were considered potentially invalid, as zero is not a meaningful value for these clinical measurements. These values were therefore treated as missing values and handled during the subsequent preprocessing steps.

5.2 Missing Value Handling

The dataset did not contain any explicitly recorded missing values. However, zero values in Glucose, BloodPressure, SkinThickness, Insulin, and BMI were considered potentially invalid for these clinical measurements. These values were replaced with missing values (NaN). The resulting missing values were later handled using median imputation during preprocessing.

5.3 Outlier Handling

Outliers were examined using the Interquartile Range (IQR) method. Insulin and SkinThickness showed relatively more extreme observations. These values were retained as they may represent valid patient measurements, and removing them could result in the loss of useful information.

5.4 Feature Engineering

We added four extra features based on existing medical knowledge, without using the target column to create them:

- **AgeGroup** groups patients into age brackets, since diabetes risk does not increase in a simple straight line with age.
- **BMICategory** groups BMI into standard categories such as underweight, normal, overweight, and obese.
- **GlucoseCategory** groups glucose readings into normal, prediabetic, and diabetic ranges.
- **Glucose_BMI_Interaction** combines Glucose and BMI into a single value, since high glucose and high BMI together may indicate a higher risk of diabetes than either feature alone.

We tested these new features and found an improvement, with an ROC-AUC of 0.8387 with the new features compared with 0.8070 without them. Therefore, we kept these features in the final pipeline.

We also compared StandardScaler and MinMaxScaler and found that MinMaxScaler gave slightly better results than StandardScaler. Therefore, MinMaxScaler was used for the final model.

5.5 Train-Test Split

The dataset was divided into training and testing sets using an 80:20 split. Stratified sampling was used to maintain a similar distribution of the two outcome classes in both sets. The training data was used to fit the preprocessing pipeline and train the models, while the test data was kept separate for final evaluation.

6 Methodology

6.1 Overall Project Workflow

Once we understood the dataset, we prepared it for modelling. Figure 6 shows the overall steps we followed, from the raw dataset to the deployed application. Each step is explained in this section and the ones that follow.

The main preprocessing steps were:

1. Checked for missing values and duplicate rows. No duplicate records or explicitly recorded missing values were found.
2. Checked zero values in the physiological measurement columns, including Glucose, BloodPressure, SkinThickness, Insulin, and BMI, as these may represent potentially invalid measurements.

3. Replaced the suspicious zero values with missing values and handled them using median imputation during preprocessing. The imputation was fitted only on the training data to avoid data leakage.
4. Added engineered features including AgeGroup, BMICategory, GlucoseLevel, Glucose_BMI, Glucose_Age, BMI_Age, and Insulin_log.
5. Split the data into 80% training and 20% testing using stratified sampling and a fixed random seed of 42, keeping the class distribution consistent and making the results reproducible.
6. Scaled the numerical features using MinMaxScaler. The categorical features created during feature engineering were encoded using OneHotEncoder before being given to the models.

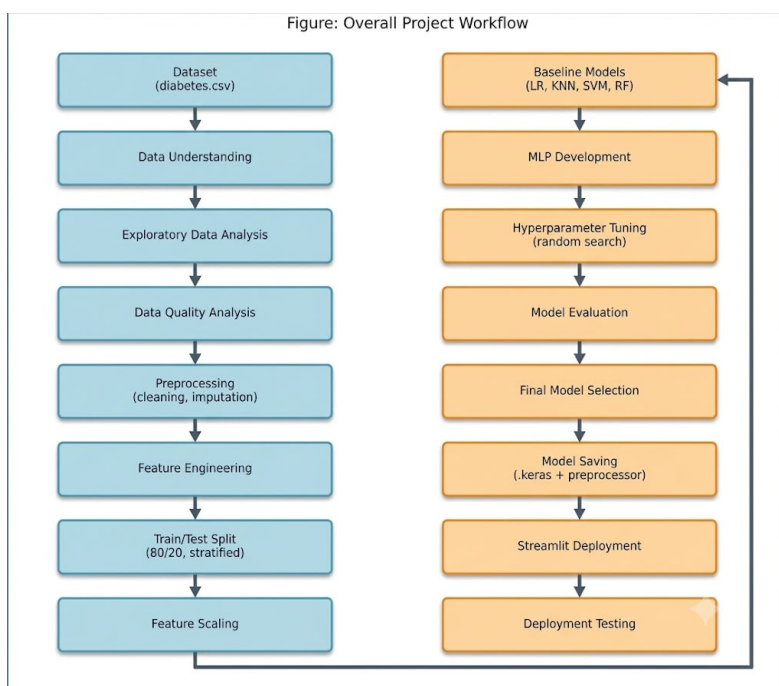


Figure 6: Overall steps followed in this project.

6.2 Baseline Models

Four traditional machine learning models were used as baseline models: **Logistic Regression**, **K-Nearest Neighbors (KNN)**, **Support Vector Machine (SVM)**, and **Random Forest**. These models were trained on the preprocessed dataset and evaluated using Accuracy, Precision, Recall, F1 Score, and ROC-AUC. Their results were then compared with the final Multilayer Perceptron (MLP) model to assess the improvement achieved by the neural network.

7 Models Used

We first trained four standard machine learning models as a baseline, and then built a Multilayer Perceptron. The idea was to see how a neural network compares against simpler, well-known models on the same problem.

- **Logistic Regression** — a simple linear model that is easy to interpret. It works well as a first baseline and shows the direction in which each feature pushes the prediction.
- **K-Nearest Neighbors (KNN)** — classifies a patient based on the class of the most similar patients in the training data. It does not really “learn” a formula; it just compares new patients to ones it has already seen.
- **Support Vector Machine (SVM)** — finds a boundary that best separates diabetic and non-diabetic patients. We used an RBF kernel, which allows the boundary to be curved instead of a straight line.
- **Random Forest** — combines many decision trees to make a more accurate and stable prediction. Each tree looks at the data a little differently, and the forest takes a majority vote.
- **Multilayer Perceptron (MLP)** — a small neural network, the main focus of this project. It learns patterns in the data through several layers of simple mathematical units connected together.

All five models were trained and tested on the exact same data split, so the comparison between them is fair. The next section focuses on how the MLP itself was built and tuned, since it needed more design decisions than the other four models.

8 MLP Development and Tuning

8.1 Architecture

Our final MLP architecture, shown in Figure 7, was designed to learn non-linear relationships between the clinical features and diabetes outcome while minimizing overfitting.

The architecture consists of the following components:

- **Input Layer:** The model receives 22 processed features after preprocessing and one-hot encoding of categorical features.
- **Hidden Layer 1:** The first fully connected layer contains 64 neurons with ReLU activation and L2 regularization.
- **Dropout Layer:** A dropout rate of 0.30 was applied to reduce overfitting.
- **Hidden Layer 2:** The second fully connected layer contains 32 neurons with ReLU activation and L2 regularization.
- **Dropout Layer:** A dropout rate of 0.20 was used as an additional regularization technique.
- **Hidden Layer 3:** The third hidden layer contains 16 neurons with ReLU activation, allowing the network to learn more complex patterns.
- **Output Layer:** A single neuron with sigmoid activation produces the probability of the positive class.
- **Training:** The model uses the Adam optimizer with binary cross-entropy loss. Early stopping was applied with a patience of 15 to retain the best validation weights.

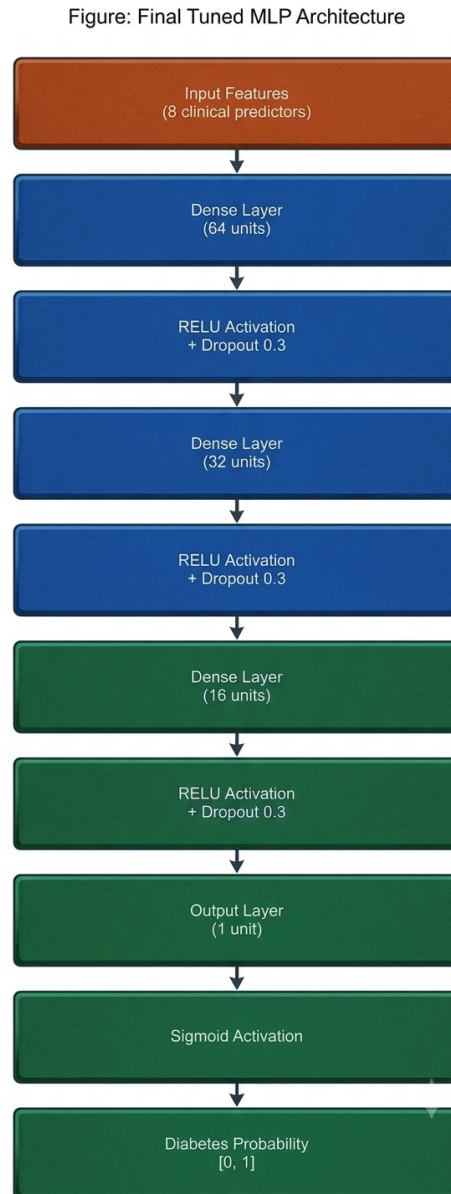


Figure 7: Final MLP architecture.

8.2 Hyperparameter Tuning

Different MLP settings were considered to obtain a suitable model configuration. The parameters included the number of hidden layers, number of neurons, learning rate, batch size, activation function, optimizer, and dropout rate. Figure ?? shows the main settings considered and the final configuration selected for the model.

The final configuration used three hidden layers with 64, 32, and 16 neurons, respectively, ReLU activation, Adam optimizer with a learning rate of 0.001, a batch size of 32, and dropout rates of 0.30 and 0.20. L2 regularization and early stopping were also applied during training.

Figure 8 and Figure 9 show the training and validation performance during the final MLP training run.

The training and validation curves remained relatively close, indicating that there was no large gap between training and validation performance.

Table 1: Hyperparameter Tuning Search Space and Selected Values

Parameter	Values Tried	Selected
Hidden Layers	1, 2, 3	3
Neurons per Layer	16, 32, 64, 128	64, 32, 16
Learning Rate	0.001, 0.0005, 0.01	0.001
Batch Size	16, 32, 64	32
Activation	ReLU, Tanh	ReLU
Optimizer	Adam, RMSprop	Adam
Dropout	0.0, 0.2, 0.3, 0.5	0.30, 0.20

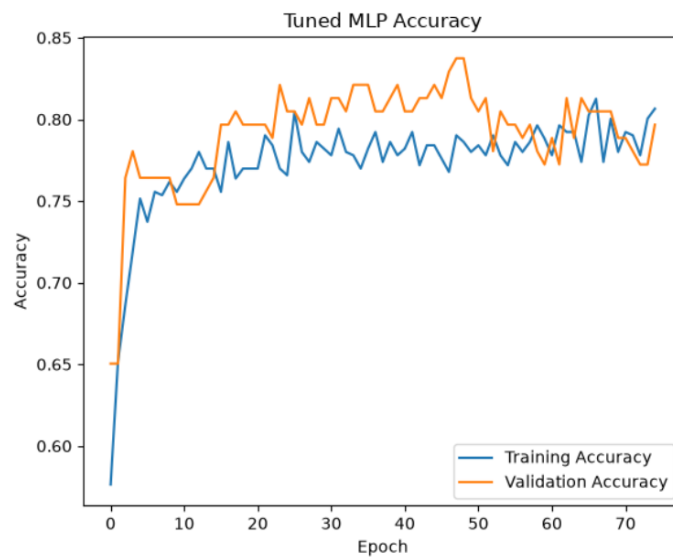


Figure 8: Training and validation loss curves for the final MLP.

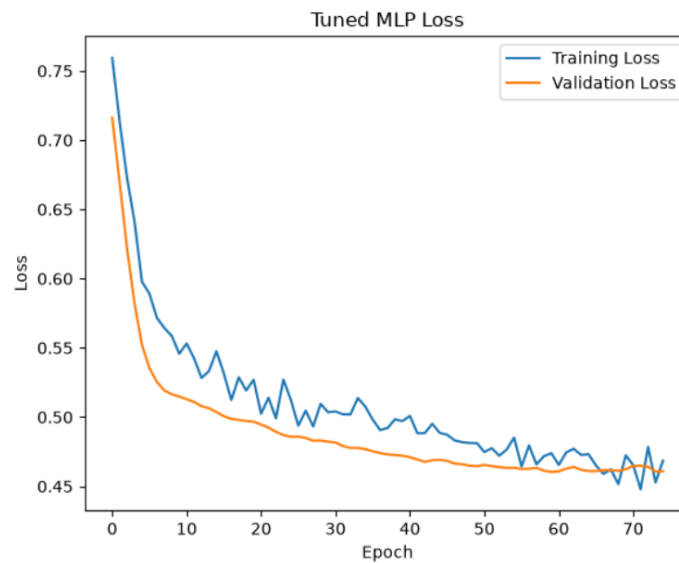


Figure 9: Training and validation ROC-AUC curves for the final MLP.

9 Model Evaluation and Comparison

Once all five models were trained, they were compared using five evaluation metrics: Accuracy, Precision, Recall, F1-score, and ROC-AUC. Accuracy measures the overall percentage of correct predictions. Precision shows how many of the patients predicted as diabetic were actually diabetic, while Recall measures how many of the actual diabetic cases were correctly identified. F1-score provides a balance between Precision and Recall. ROC-AUC measures how well the model distinguishes between the two classes across different classification thresholds.

Accuracy was not considered alone because approximately 65% of the patients in the dataset were non-diabetic. Therefore, a model that always predicted the majority class could still achieve around 65% accuracy without effectively identifying diabetic cases. Hence, multiple evaluation metrics were considered to provide a more complete comparison of the models.

Table 2: Model Performance Comparison

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	Training Time (s)
Logistic Regression	0.734	0.633	0.574	0.602	0.839	0.029
K-Nearest Neighbors	0.766	0.688	0.611	0.647	0.841	0.208
Support Vector Machine	0.740	0.646	0.574	0.608	0.809	0.111
Random Forest	0.753	0.629	0.722	0.672	0.823	1.221
Multilayer Perceptron	0.80	0.717	0.704	0.710	0.850	7.027

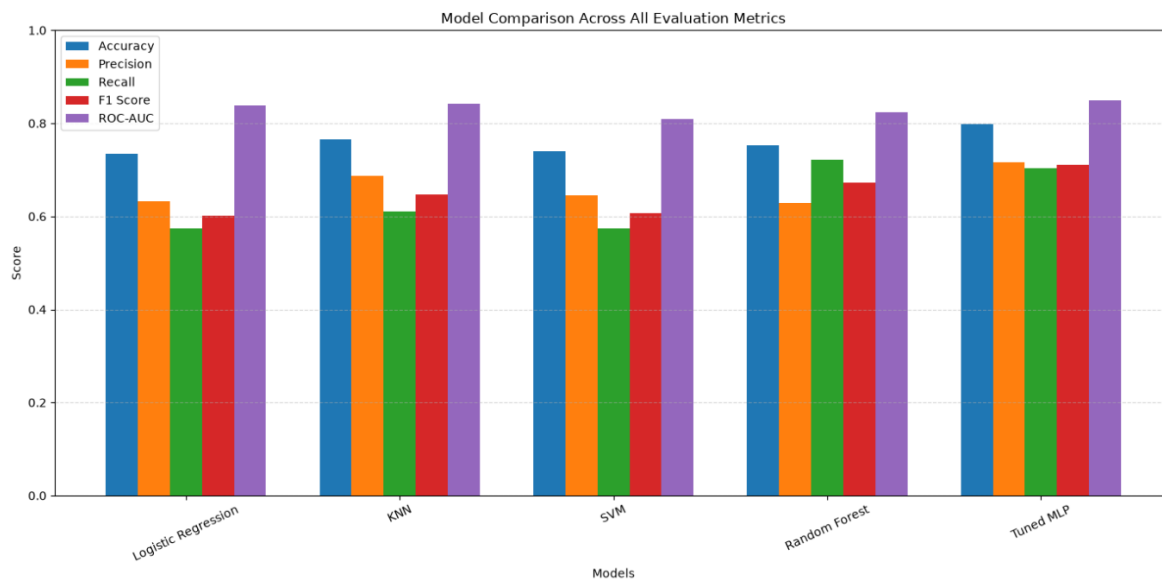


Figure 10: Model comparison across all five evaluation metrics.

Figure 10 compares the performance of all five models across Accuracy, Precision, Recall, F1-score, and ROC-AUC on the test set. The comparison provides an overall view of the predictive performance of the models.

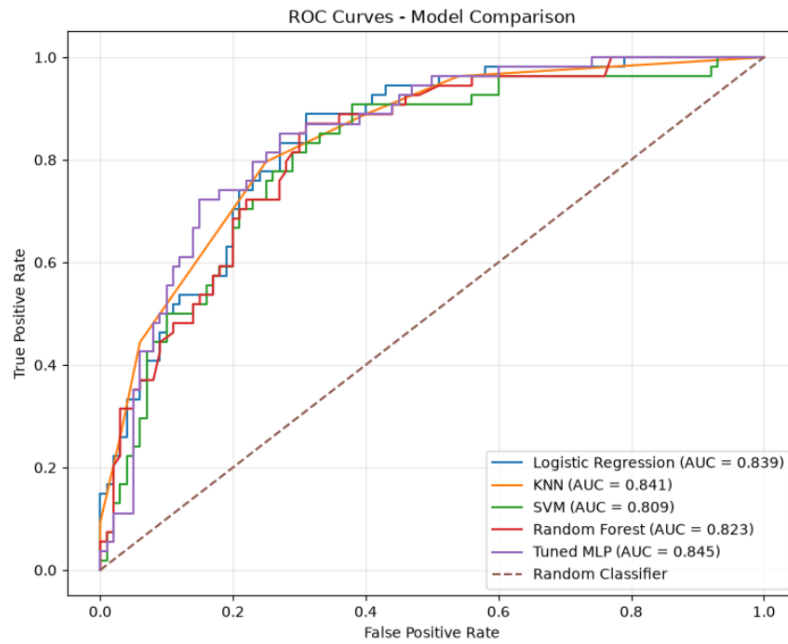


Figure 11: ROC curves for all five models.

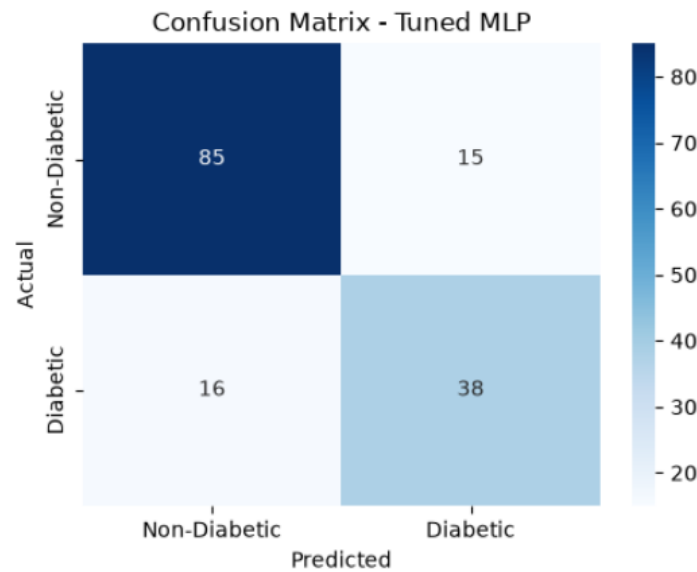


Figure 12: Confusion matrix of the final MLP on the test set.

10 Final Model

The Tuned MLP achieved the highest test accuracy of **79.87%**, followed by KNN with **76.62%**. The MLP also achieved the highest ROC-AUC (**0.8496**) and F1-score (**0.7103**). Its confusion matrix showed **16 false-negative** and **15 false-positive** cases. Based on the overall results in the model comparison, we selected the **Tuned MLP as the final model**. The trained MLP and preprocessing pipeline were saved for reuse in predictions on new patient data.

The trained MLP (`diabetes_mlp.keras`) and preprocessing pipeline (`diabetes_preprocessor.joblib`) were saved for direct reuse.

11 Model Deployment and Testing

11.1 Deployment

We built a simple web application using Streamlit so that anyone can try the model. The app loads the saved MLP model and the saved preprocessing pipeline, takes the same eight input values used during training, and shows the predicted class (Diabetic / Non-Diabetic) along with the predicted probability. Basic input checks are included for example, the app will not accept a Glucose value of 0, since that is not realistic.

11.2 Deployment Testing

We tested the application with a few different cases to make sure it works correctly. We also checked that the prediction from the deployed application matched the prediction obtained from our Python code for the same input.

Figure 13: *
(a) App home screen

Figure 14: *
(b) A diabetic prediction

Figure 15: The deployed Streamlit application.

11.3 Testing

We tested the app with a few different cases to make sure it works correctly, and also checked that the app's prediction matched the prediction from our own Python code for the same input.

Table 3: Deployment Test Cases

Test Case	Scenario	Result
Likely non-diabetic	Low glucose and BMI	Non-diabetic, 0.17%
Likely diabetic	High glucose and BMI	Diabetic, 78.11%
Borderline case	Values close to the decision boundary	Diabetic, 56.45%
Invalid input	Glucose entered as 0	App showed an error and did not predict
Unexpected input	Very high values in several fields	App still predicted, no crash

12 Limitations

Like any student project, this one has some limitations that are important to keep in mind.

- The dataset is quite small — only 768 records in total, which limits how confident we can be in the exact numbers.
- There is some class imbalance (more non-diabetic than diabetic cases), which is why we did not rely on accuracy alone.
- All patients in the dataset are female and of Pima Indian heritage, so the model may not work well for other groups such as men or people from different backgrounds.
- Some zero values in the original data may actually represent missing measurements that were already filled in before we received the file, and we cannot recover the original values.
- Because the dataset is small, results could change slightly with a different train-test split or a different random seed.
- This model is for academic learning only — it is not a medical diagnostic tool and must not be used to make real healthcare decisions.

13 Conclusion and Future Scope

13.1 Conclusion

In this project, we built a complete pipeline to predict diabetes using the Pima Indians Diabetes dataset — starting from understanding the raw data, all the way to a working, tested web application. We compared four traditional machine learning models with a tuned Multilayer Perceptron, using accuracy, precision, recall, F1-score and ROC-AUC instead of accuracy alone. The Tuned MLP and KNN gave the best results based on accuracy, but we picked the MLP as our final model since it had the highest accuracy, ROC-AUC and F1-score among the five models.

We deployed the final model using a Streamlit web application that takes patient details and returns a prediction with its probability, and we tested it with several real and edge-case inputs to make sure it behaves correctly. This system was developed for academic and educational purposes and should not be used as a standalone medical diagnostic system.

13.2 Future Scope

- Test the model on a larger and more varied dataset.
- Include male patients and other age groups to see if the results still hold.
- Try a few additional models, such as Gradient Boosting.
- Improve the Streamlit app with a nicer interface and more input checks.
- Study which features matter most in more detail.

14 References

References

- [1] UCI Machine Learning Repository (mirrored by Kaggle user uciml), “Pima Indians diabetes database,” 2016, accessed 2026. Available at: <https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database>
- [2] J. W. Smith, J. E. Everhart, W. C. Dickson, W. C. Knowler, and R. S. Johannes, “Using the ADAP learning algorithm to forecast the onset of diabetes mellitus,” in *Proceedings of the Symposium on Computer Applications and Medical Care*. IEEE Computer Society Press, 1988, pp. 261–265.
- [3] Pedregosa, F. et al. (2024) “Scikit-learn: Machine learning in Python.” [Online]. Available: <https://scikit-learn.org/stable/documentation.html>
- [4] Abadi, M. et al. (2024) “TensorFlow / Keras API Documentation.” [Online]. Available: https://www.tensorflow.org/api_docs
- [5] Streamlit Inc. (2024) “Streamlit documentation.” [Online]. Available: <https://docs.streamlit.io/>
- [6] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [7] N. V. Chawla, K. W. Bowyer, L. O. Hall, and P. W. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [8] American Diabetes Association, “2. Classification and diagnosis of diabetes: Standards of medical care in diabetes,” *Diabetes Care*, vol. 42, no. Supplement_1, pp. S13–S28, 2019.

A Appendix

The complete source code, Jupyter notebook, trained model, preprocessing pipeline, and Streamlit application are included with the project submission.

A Contributions of Group Members

Sl.	Group Member	Regd. No.	Role / Responsibility	Contribution (%)
1	Subhashree Mohanty	2341013309	Data Preprocessing, feature engineering, baseline model development	25%
2	Manish Kumar Baitharu	2341002004	MLP development, hyperparameter tuning, model evaluation	25%
3	Omm Prakash Nayak	2341014065	Streamlit deployment, visualization, and documentation	25%
4	Ayushman Sahoo	2341019113	Dataset understanding, EDA, project integration	25%

All four members took part in integration, cross-review, and final proofreading of the report.